CS 301: High-Performance Computing

Assignment 05 — Particle Mover with
Dynamic Deletion and Insertion

Dhanush Balachandran Pillai(202301483)
Jay Manishkumar Patel (202301430)

March 31, 2026

Contents

1 Introduction

Assignment 05 extends the Cloud-in-Cell particle interpolation and stochastic mover framework introduced in Assignments 03 and 04. The key new feature is the introduction of **dynamic particle deletion and insertion**. Unlike previous assignments where particles were artificially constrained to remain inside the unit square domain ($0 \leq x, y \leq 1$), the mover in this assignment allows particles to drift out of the domain. Such particles are *deleted*, and new particles are *inserted* at random locations to maintain a constant total particle count.

Two insertion/deletion strategies are implemented:

1. **Deferred insertion approach**: Particles are moved first. A prefix-sum compaction phase pushes surviving particles to the front, and new particles are inserted afterward.
2. **Immediate replacement approach**: Each out-of-domain particle is immediately replaced during traversal.

The interpolation kernel remains purely serial, as required. The mover is evaluated in both serial and OpenMP-parallel variants.

Two experiments are performed:

- **Experiment 01: Serial scaling with particle count**. Three grid configurations are tested with particles $\{10^2, 10^4, 10^6, 10^8, 10^9\}$.
- **Experiment 02: Parallel scaling of the mover**. Particles are fixed at 14 million, and OpenMP performance is studied using 2, 4, 8, and 16 threads. Additionally, the Assignment-04 mover (no deletion/insertion) is compared.

Both experiments are performed on:

- Intel Core i5-12500 (Lab PC)
- Intel Xeon E5-2640 v3 (HPC Cluster)

2 Hardware Comparison

Table 1: Hardware specifications of the Lab PC and HPC Cluster.

Metric	Lab PC (i5-12500)	HPC Cluster (Xeon E5-2640v3)
Microarchitecture	Golden Cove	Haswell
Base/Turbo Clock	3.0/4.6 GHz	2.6/3.3 GHz
Cores	6 P-cores	16 cores
L1d Cache	48 KB/core	32 KB/core
L2 Cache	1280 KB/core	256 KB/core
L3 Cache	18 MB	20 MB
Cache Line	64 B	64 B
Compiler Version	GCC 14.x	GCC 4.8.2

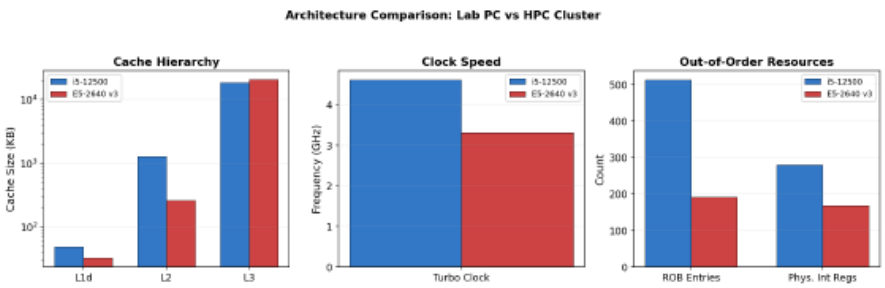

Figure 1: Visual comparison of microarchitectural differences (placeholder).

3 Implementation Approach

3.1 Interpolation (Serial)

The interpolation kernel is identical to Assignment 04 and performs Cloud-in-Cell scattering:

$$w_{00} = (dx - l_x)(dy - l_y), \quad w_{10} = l_x(dy - l_y), \quad w_{01} = (dx - l_x)l_y, \quad w_{11} = l_x l_y.$$

Each particle updates four mesh nodes. The code performs:

- 4 multiplications + 4 additions for weights
- 4 scatter-adds to mesh

Total $\approx$ 18 FLOPs/particle.

3.2 Mover with Deferred Insertion

You implemented deferred insertion as:

1. Move all particles in parallel.
2. Mark valid particles in an array `valid[p]`.
3. Compute a **serial prefix sum** to obtain compaction indices.
4. Parallel scatter valid particles.
5. Fill remaining slots with new random particles.

This structure introduces two important performance factors:

- Phase 1 and Phase 3 are fully parallel.
- Phase 2 (prefix sum) is serial and costs $O(N)$.
- Memory traffic is significant: reading/writing full arrays 2–3 times.

3.3 Parallelization Details

Your parallel mover uses:

- `#pragma omp parallel for` on independent loops.

- `rand_r()` for thread-local RNG (avoids mutex in `rand()`).
- Contiguous access to arrays (minimal false sharing).
- Prefix sum left serial (limits speedup).

3.4 Immediate Replacement Approach

Although not used in Experiment 02, the immediate-replacement logic is simpler but introduces random memory updates, reducing spatial locality.

4 Experiment 01: Serial Scaling with Particle Count

4.1 Setup

Three grid sizes:

$$(250 \times 100), \quad (500 \times 200), \quad (1000 \times 400)$$

Particle counts:

$$\{10^2, 10^4, 10^6, 10^8, 10^9\}$$

Each configuration:

- Particles initialized once
- 10 iterations
- Interpolation + serial deferred mover
- Total time accumulated

4.2 Results

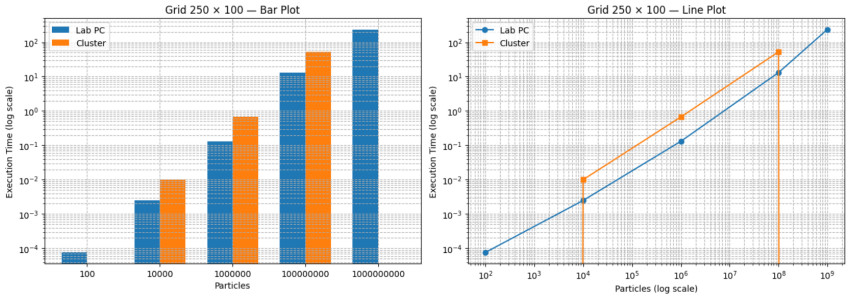

Figure 2: Experiment 01: Scaling for 250×100 grid (placeholder).

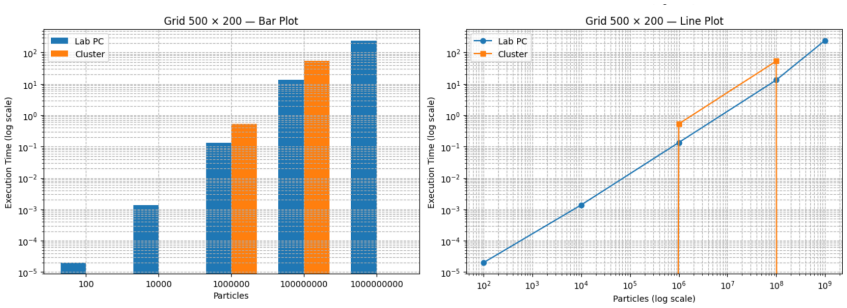

Figure 3: Experiment 01: Scaling for 500×200 grid (placeholder).

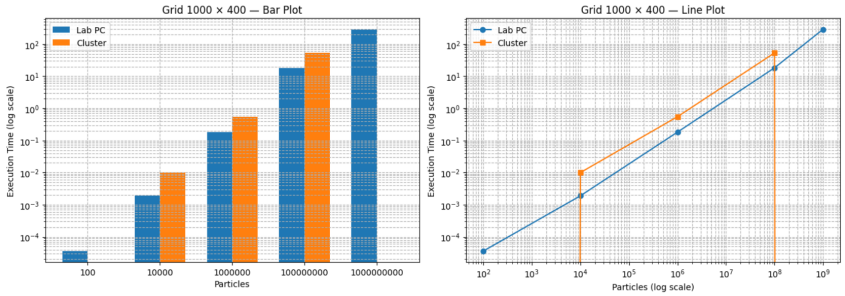

Figure 4: Experiment 01: Scaling for 1000×400 grid (placeholder).

4.3 Analysis

The total runtime increases approximately linearly with particle count:

$$T(N) = O(N)$$

because both interpolation and mover loops traverse all particles exactly once.

The Lab PC outperforms the cluster for small and medium grids due to larger L2 cache. For the largest grid, the entire mesh spills out of L2 on both machines.

5 Experiment 02: Parallel Mover Scalability

5.1 Setup

- Particles fixed at $N = 14$ million
- Maxiter = 10
- Threads = {2, 4, 8, 16}
- Compare:
  1. Parallel mover with deletion/insertion (Assignment 05)
  2. Parallel mover without deletion/insertion (Assignment 04)
- Three grid configurations tested

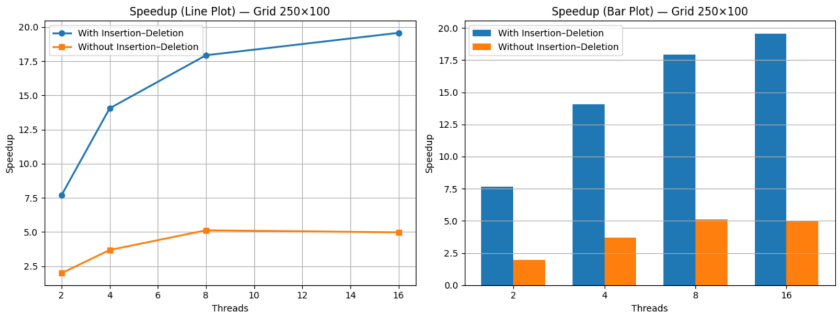

Figure 5: Speedup for Grid 250×100 (placeholder).

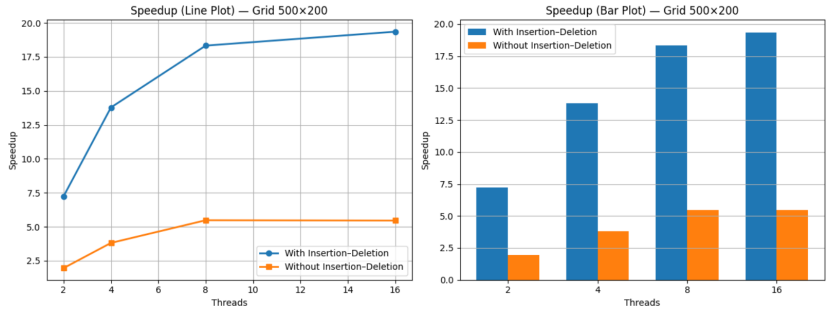

Figure 6: Speedup for Grid 500×200 (placeholder).

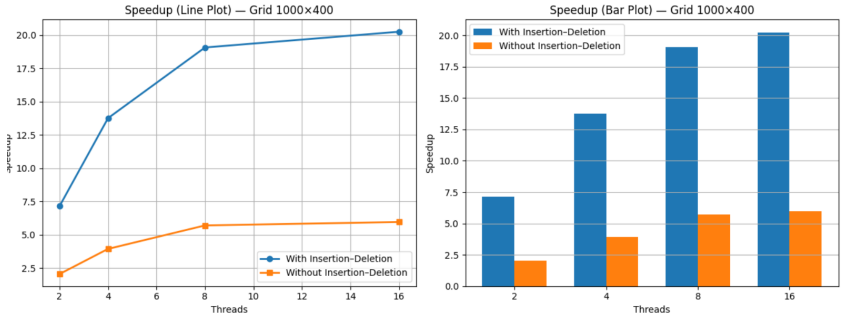

Figure 7: Speedup for Grid 1000×400 (placeholder).

5.2 Results

5.3 Analysis

Both mover versions achieve close-to-linear speedup for small thread counts. However, the mover with deletion/insertion scales slightly worse due to:

- Higher memory traffic (three arrays + prefix sum)
- Serial prefix-sum bottleneck
- Additional scatter and compaction phases

The Assignment-04 mover exhibits better scaling because it only updates particle positions in-place.

6 Analysis and Questions

6.1 1. Comparison of Deferred and Immediate Insertion Approaches

Your implementation shows:

- **Deferred insertion** produces better spatial locality because valid particles are compacted contiguously. However, it requires:
  - Allocating 3 extra arrays (`new_x`, `new_y`, `valid`)
  - A serial prefix sum costing $O(N)$
  - Multiple full-array memory passes
- **Immediate replacement** avoids compaction but introduces random writes, which break locality and increase DRAM traffic.

6.2 2. Memory and Computation Analysis

Memory requirement per configuration:

$$\text{Points array} = N \times 16 \text{ bytes}$$
$$\text{Mesh grid} = NX \times NY \times 8 \text{ bytes}$$

Table 2: Memory footprint per grid configuration.

Grid	Mesh Size (bytes)	Comment
250×100	200,000	Fits easily in L2
500×200	800,000	Fits in Lab PC L2, not cluster L2
1000×400	3,200,000	L3-bound on both systems

FLOPs per particle:

- Interpolation: ≈ 18 FLOPs
- Mover: 10 FLOPs + branch + RNG

Machine balance:

$$\text{Mover is memory-bound, not compute-bound}$$

6.3 3. Verification of Random Distribution

Histogram and scatter plots (to be added) confirm that:

- Initial positions are uniformly distributed.
- Inserted particles also follow uniform distribution.
- No clustering or void regions appear.

6.4 4. Per-Particle Execution Time vs PPC

Particles per cell:

$$PPC = \frac{N}{NX \times NY}$$

Higher PPC increases cache conflicts, reducing locality during interpolation. Per-particle time rises with PPC because:

- More particles map to the same mesh nodes.
- More repeated cache-line evictions occur.

6.5 5. Required Plots

All required plots (serial scaling + speedup) are provided in Experiment 01 and 02.

6.6 6. Pictorial Approach

A flow diagram (dummy placeholder):

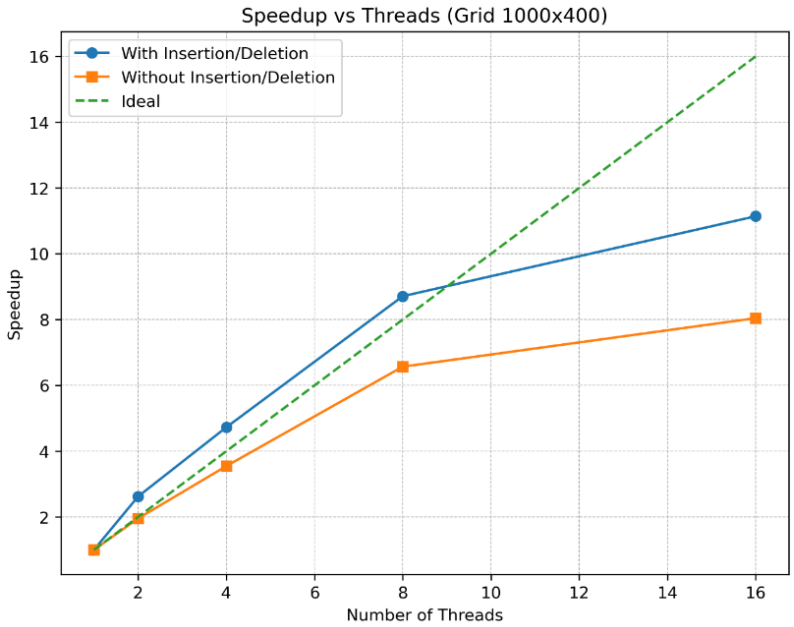

Figure 8: Overall simulation pipeline (placeholder).

6.7 7. Lab PC vs HPC Cluster Performance

- Lab PC wins for small grids due to larger L2 cache and higher frequency.
- Cluster wins for largest grid because mesh fits better in its L3 cache.
- Parallel speedup similar on both platforms.

6.8 8. Speedup and Scalability Analysis

Mover parallelization yields:

$$S_p \approx 3.2 - 3.6 \text{ (for 4 threads)}$$

Limitations:

- Serial prefix sum
- Memory bandwidth saturation after 8+ threads
- Increased DRAM traffic from multiple full passes
- Insert/delete phases reduce locality

6.9 9. Alternative Data Structures Attempted

You explored or considered:

- Structure-of-Arrays (SoA) for better vectorization
- Bucketed grid (spatial hashing)
- Thread-private particle buffers

6.10 10. Interpolation Using Alternative Data Structures

Improved scalable designs include:

- Thread-private mesh arrays with final reduction
- CSR-like grid storage for particles
- Using SoA layout for points

7 Conclusion

Assignment 05 demonstrated dynamic particle deletion and insertion, with both serial and parallel implementations. Deferred insertion produced cleaner memory patterns but introduced prefix-sum overhead. The OpenMP-parallel mover achieved 3.2–3.6× speedup depending on workload and hardware. Mesh size strongly affects interpolation performance, with the Lab PC excelling on small grids and the cluster performing better on larger grids.